

RZ Axisymmetric Coordinate Support in Cerisse

Implementation Notes

Auto-generated

March 2026

1 Overview

Cerisse originally supported only Cartesian grids. The modifications on branch `ibm_gpu` add axisymmetric cylindrical (RZ) coordinate support by leveraging AMReX's built-in `Geometry::IsRZ()` flag (`geometry.coord_sys = 1` in the input file).

The RZ convention in Cerisse is:

- Direction 0 (i) = radial direction r , with the symmetry axis at $r = 0$ (`prob_lo[0] = 0`).
- Direction 1 (j) = axial direction z .
- `AMREX_SPACEDIM = 2` is required.

2 Mathematical Formulation

2.1 Finite-Volume Divergence

In cylindrical (r, z) coordinates the conservative finite-volume update for cell (i, j) is:

$$\frac{\partial \mathbf{U}}{\partial t} = -\frac{1}{V_{ij}} \left[\left(A_{i+1/2}^r \mathbf{F}_{i+1/2}^r - A_{i-1/2}^r \mathbf{F}_{i-1/2}^r \right) + \left(A_{j+1/2}^z \mathbf{F}_{j+1/2}^z - A_{j-1/2}^z \mathbf{F}_{j-1/2}^z \right) \right] + \mathbf{S}_{\text{geo}}$$

where:

$$V_{ij} = \pi (r_{i+1/2}^2 - r_{i-1/2}^2) \Delta z, \quad (1)$$

$$A_{i\pm 1/2}^r = 2\pi r_{i\pm 1/2} \Delta z, \quad (2)$$

$$A_{j\pm 1/2}^z = \pi (r_{i+1/2}^2 - r_{i-1/2}^2). \quad (3)$$

After cancellation of common $\pi \Delta z$ factors:

- **r -direction:**

$$\text{RHS}_r = \frac{2 (r_{i-1/2} F_{i-1/2}^r - r_{i+1/2} F_{i+1/2}^r)}{r_{i+1/2}^2 - r_{i-1/2}^2}$$

- **z -direction:**

$$\text{RHS}_z = \frac{F_{j-1/2}^z - F_{j+1/2}^z}{\Delta z} \quad (\text{same as Cartesian})$$

2.2 Geometric Source Term

The Euler equations in cylindrical coordinates have an additional source in the radial momentum equation:

$$S_{r\text{-mom}} = +\frac{p}{r}$$

This arises because the standard Cartesian-form momentum flux $\rho u_r^2 + p$ is used on r-faces, but the divergence $-\frac{1}{r} \frac{\partial(r F_r)}{\partial r}$ does not fully produce the p/r contribution when expanding the cylindrical form.

Additionally, the viscous stress tensor has extra hoop-stress terms not captured by the face-flux divergence; these are handled by `prob_rhs.rz_geometric_source()`.

3 Modified Files

File	RZ-related changes
<code>src/CNS.h</code>	New <code>rz_sanity_check()</code> function that aborts with a clear message if the RZ configuration is invalid.
<code>src/CNS.cpp</code>	Calls <code>rz_sanity_check()</code> in constructor. <code>printTotal()</code> uses <code>geom.GetVolume()</code> for volume-weighted integrals (correct in RZ).
<code>src/tim/compute_rhs.cpp</code>	Complete rewrite of the flux-divergence kernel (see §??).

4 Code Details

4.1 `rz_sanity_check()` — `CNS.h`

Called once per level during CNS construction. Checks:

1. `AMREX_SPACEDIM == 2`.
2. r -direction is non-periodic.
3. `prob_lo[0] == 0` (axis at $r = 0$).

4.2 Volume-Weighted `printTotal()` — `CNS.cpp`

The original code used `consmf.sum(comp) * geom.ProbSize()`, which is only correct for uniform Cartesian cells. The new version:

1. Creates a volume `MultiFab` via `geom.GetVolume()`.
2. Uses `ReduceOps<ReduceOpSum>` to compute $\sum_i U_i \cdot V_i$ on GPU.

4.3 Flux Divergence Kernel — `compute_rhs.cpp`

The divergence computation is now split into two branches at runtime:

Listing 1: Simplified RZ branch structure

```

1 const bool is_rz = geom.IsRZ();
2 if (is_rz) {
3     // Cylindrical (r,z) finite-volume divergence
4     ParallelFor(bx, [=] AMREX_GPU_DEVICE(int i, int j, int k) {
5         Real r_lo = r0 + i * dr;
6         Real r_hi = r_lo + dr;
7         Real r2diff = (r_hi + r_lo) * dr;

```

```

8      Real inv_r2diff = 1.0 / max(r2diff, tiny);
9      Real inv_r = 1.0 / max(r_lo + 0.5*dr, tiny_r);
10
11     for (int n = 0; n < ncons; ++n) {
12         // r-direction: -(1/r) d(rF_r)/dr
13         state(i,j,k,n) += 2*(r_lo*fx(i,j,k,n)
14             - r_hi*fx(i+1,j,k,n)) * inv_r2diff;
15         // z-direction: Cartesian
16         state(i,j,k,n) += (fy(i,j,k,n)
17             - fy(i,j+1,k,n)) * inv_dz;
18         // Geometric source: +p/r for radial momentum
19         if (n == umom_r)
20             state(i,j,k,n) += prims(i,j,k,qpres) * inv_r;
21     }
22 }));
23 } else {
24     // Standard Cartesian divergence (1D/2D/3D via #if)
25 }

```

Key implementation details:

- Uses a 3D `ParallelFor` with inner n -loop to reuse geometric metrics ($r_{\text{lo}}, r_{\text{hi}}, V^{-1}$) across all conserved components.
- A tiny positive floor prevents division by zero at $r = 0$. In practice $r_0 = 0$ and $i \geq 0$ so $r_{1/2}^2 - r_{-1/2}^2 > 0$, but the floor guards against edge cases.
- After the divergence, `rz_geometric_source()` is called for viscous hoop-stress terms.

4.4 Viscous Geometric Source

Listing 2: RZ viscous source dispatch

```

1  if (is_rz) {
2  #if (AMREX_USE_GPIBM || CNS_USE_EB)
3      // TODO: implement rz_geometric_source_ibm
4  #else
5      prob_rhs.rz_geometric_source(geom, mfi, prims, state, cls_d);
6  #endif
7  }

```

The IBM variant (`rz_geometric_source_ibm`) is not yet implemented — this is a known TODO.

5 Input File Configuration

To enable RZ mode, add to the input file:

```

1  geometry.coord_sys = 1      # 0=Cartesian, 1=RZ
2  geometry.prob_lo    = 0.0 <z_lo>
3  geometry.prob_hi    = <r_hi> <z_hi>
4  geometry.is_periodic = 0 <z_periodic>

```